

1

a

i

ii

iii

iv

b

i

ii

iii

iv

2

a

$$x = 2x - yx^2$$

$$\rightarrow yx^2 + x = 2x$$

$$\rightarrow yx^2 = x$$

$$\rightarrow yx = 1$$

$$\rightarrow x = \frac{1}{y}$$

$$\text{Or alternatively, } f\left(\frac{1}{y}\right) = \frac{2}{y} - \frac{y^2}{y} = \frac{2}{y} - \frac{1}{y} = \frac{1}{y}$$

b

Relative error $e_n = \left| \frac{\frac{1}{y} - x_n}{\frac{1}{y}} \right| = |1 - yx_n|$

And e_{n+1} will be $|1 - yx_{n+1}|$ so

$$\rightarrow e_{n+1} = |1 - y(2x_n - yx_n^2)|$$

$$\rightarrow e_{n+1} = |1 - 2yx_n + y^2x_n^2|$$

$$\rightarrow e_{n+1} = |(1 - yx_n)^2|$$

$$\rightarrow e_{n+1} = (e_n)^2$$

Therefore we only converge when our relative error e_n is less than 1 because squaring decreases for decimal points, stays same at 1 and goes to infinity at greater than 1.

Note that this is for both positive and negative values

Therefore if $|1 - yx| < 1$ then we will converge. Thus the range of starting x is $0 < x_0 < \frac{2}{y}$

c

We have the exact case of quadratic convergence here because the relative error at each step is approximately the square of the relative error of the previous steps. So by the notes, the number of correct decimal digits doubles with each iteration.

d

```
import numpy as np
import matplotlib.pyplot as plt

def reciprocal(y, x0, tol=1e-15, max_iter=100):
    """Division-free iteration  $x_{n+1} = 2x_n - yx_n^2$  for  $1/y$ ."""
    xs = [float(x0)]
    for _ in range(max_iter):
        x_new = 2 * xs[-1] - y * xs[-1] * xs[-1]
        xs.append(x_new)
        if not np.isfinite(x_new) or abs(x_new) > 1e100:
            break # diverged
        if abs(xs[-1] - xs[-2]) < tol * max(1.0, abs(x_new)):
            break
```

```

    return np.array(xs)

# ---- Figures ----
fig, axs = plt.subplots(2, 2, figsize=(12, 9))
ax = axs.ravel()

# Overlapping curves are drawn with different markers, line styles and
# decreasing marker sizes, plus some transparency, so every curve stays visible.
markers      = ["o", "s", "^", "D", "v", "x", "+"]
linestyles    = ["-", "--", "-.", ":", (0, (5, 1)), (0, (3, 1, 1, 1)), (0, (1, 1))]
def style(i, n):
    return dict(marker=markers[i % 7], ls=linestyles[i % 7], ms=10 - 6 * i / max(n - 1, 1),
                mfc="none", mew=1.5, lw=2, alpha=0.8)

# (1) value of x_n vs n, several x0 for y = 3
y = 3.0
x0s = [0.01, 0.2, 0.5, 0.65, 0.66, 0.7]
for i, x0 in enumerate(x0s):
    xs = reciprocal(y, x0, max_iter=25)
    ax[0].plot(np.clip(xs, -1, 1), label=f"x0 = {x0}", **style(i, len(x0s)))
ax[0].axhline(1 / y, color="k", ls=":", label="1/y")
ax[0].set_ylim(-1.05, 1); ax[0].set_xlabel("iteration n"); ax[0].set_ylabel("x_n (clipped to")
ax[0].set_title("y = 3: converges iff 0 < x0 < 2/3"); ax[0].legend(fontsize=8)

# (2) |x_{n+1} - x_n| vs n (log scale), different y with x0 = 0.1/y
ys = [0.5, 3, 50, 1000]
for i, y in enumerate(ys):
    xs = reciprocal(y, 0.1 / y)
    ax[1].semilogy(np.abs(np.diff(xs)) * y + 1e-300, label=f"y = {y}", **style(i, len(ys)))
ax[1].set_xlabel("iteration n"); ax[1].set_ylabel("y · |x_{n+1} - x_n| (scaled step)")
ax[1].set_ylim(1e-17, 2); ax[1].set_title("x0 · y = 0.1: all 4 curves coincide (depend only on")
ax[1].legend(fontsize=8)

# (3) relative error vs n; pairs symmetric about x0*y = 1 have the same |e_0|
y = 3.0
pairs = [(0.001, 1.999), (0.1, 1.9), (0.5, 1.5)]
colors = ["tab:blue", "tab:orange", "tab:green"]
for c, (r1, r2) in zip(colors, pairs):
    for j, r in enumerate((r1, r2)):
        xs = reciprocal(y, r / y)
        ax[2].semilogy(np.abs(1 - y * xs) + 1e-300, color=c,
                        marker="o" if j == 0 else "x", ls="--" if j == 0 else "--",

```

```

        ms=10 if j == 0 else 6, mfc="none", mew=1.5, lw=2, alpha=0.8,
        label=f"x0·y = {r}  (|e0| = {abs(1 - r):.3g})")
ax[2].set_ylim(1e-17, 2); ax[2].set_xlabel("iteration n"); ax[2].set_ylabel("|1 - y x_n| (re)
ax[2].set_title("same |1 - x0 y|  same curve; slow start when |e0|  1")
ax[2].legend(fontsize=8, title="x0·y = 1 is exact after 0 steps", title_fontsize=8)

# (4) iterations to converge as a function of x0*y
r_grid = np.linspace(0.001, 1.999, 400)
iters = [len(reciprocal(1.0, r)) - 1 for r in r_grid]
ax[3].plot(r_grid, iters)
ax[3].set_xlabel("x0 · y"); ax[3].set_ylabel("iterations to converge")
ax[3].set_title("Iterations needed vs. starting value (any y)")
plt.tight_layout()

# ---- Table: does it converge for various (y, x0)? ----
print(f"{'y':>6} {'x0':>8} {'x0*y':>6} {'iters':>6} {'final x':>14} {'1/y':>14} {'result':>14}")
cases = [(3, 0.01), (3, 0.3), (3, 0.6), (3, 0.66), (3, 0.6667), (3, 0.7), (3, -0.1),
        (0.25, 0.1), (0.25, 7.9), (0.25, 8.1), (100, 0.0001), (100, 0.019), (100, 0.021)]
for y, x0 in cases:
    xs = reciprocal(y, x0)
    final = xs[-1]
    ok = np.isfinite(final) and abs(final - 1 / y) < 1e-12 * (1 / y)
    print(f"{'y':>6g} {'x0':>8g} {'x0*y':>6.3g} {'len(xs)-1':>6} {'final':>14.6e} {'1/y':>14.6e} {'converge":

# ---- Quadratic convergence check: e_{n+1} / e_n^2 should equal y ----
y, x0 = 3.0, 0.05
xs = reciprocal(y, x0)
err = np.abs(xs - 1 / y)
print(f"\ny = {y}, x0 = {x0}")
print(f"{'n':>3} {'x_n':>20} {'|x_n - 1/y|':>14} {'|x_{n+1} - x_n|':>14} {'e_{n+1} / e_n^2':>14}")
for n in range(len(xs)):
    step = abs(xs[n + 1] - xs[n]) if n + 1 < len(xs) else np.nan
    ratio = err[n + 1] / err[n] ** 2 if n + 1 < len(xs) and err[n] > 0 and err[n + 1] > 0 else
    print(f"{'n':>3} {'xs[n]':>20.16f} {'err[n]':>14.3e} {'step':>14.3e} {'ratio':>14.4f}")

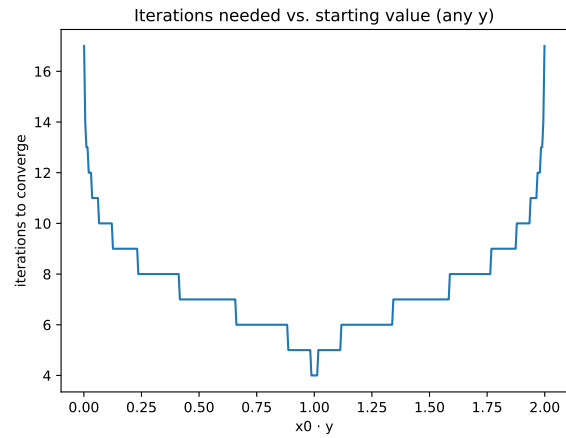
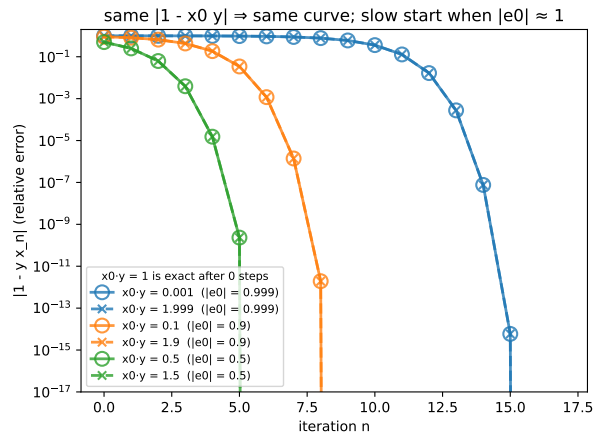
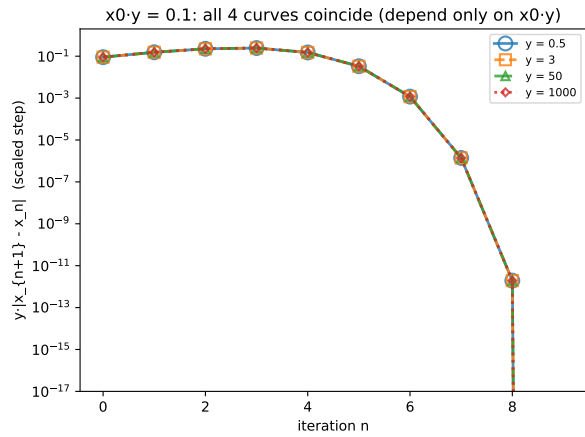
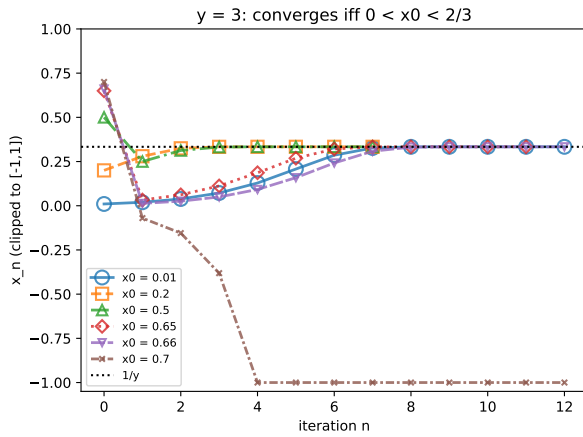
```

y	x0	x0*y	iters	final x	1/y	result
3	0.01	0.03	12	3.333333e-01	3.333333e-01	converged
3	0.3	0.9	5	3.333333e-01	3.333333e-01	converged
3	0.6	1.8	9	3.333333e-01	3.333333e-01	converged
3	0.66	1.98	12	3.333333e-01	3.333333e-01	converged
3	0.6667	2	22	-4.678211e+181	3.333333e-01	DIVERGED
3	0.7	2.1	12	-1.167662e+169	3.333333e-01	DIVERGED

3	-0.1	-0.3	10	-1.588077e+116	3.333333e-01	DIVERGED
0.25	0.1	0.025	12	4.000000e+00	4.000000e+00	converged
0.25	7.9	1.98	12	4.000000e+00	4.000000e+00	converged
0.25	8.1	2.02	14	-2.003875e+176	4.000000e+00	DIVERGED
100	0.0001	0.01	13	1.000000e-02	1.000000e-02	converged
100	0.019	1.9	10	1.000000e-02	1.000000e-02	converged
100	0.021	2.1	12	-3.502986e+167	1.000000e-02	DIVERGED

y = 3.0, x0 = 0.05

n	x_n	x_n - 1/y	x_{n+1} - x_n	e_{n+1} / e_n^2
0	0.05000000000000000	2.833e-01	4.250e-02	3.0000
1	0.09250000000000000	2.408e-01	6.683e-02	3.0000
2	0.15933125000000000	1.740e-01	8.317e-02	3.0000
3	0.2425031583203125	9.083e-02	6.608e-02	3.0000
4	0.3085829712546454	2.475e-02	2.291e-02	3.0000
5	0.3314955920642549	1.838e-03	1.828e-03	3.0000
6	0.3333232014544171	1.013e-05	1.013e-05	3.0000
7	0.3333333330253684	3.080e-10	3.080e-10	585.2991
8	0.3333333333333334	5.551e-17	0.000e+00	18014398509481984.0000
9	0.3333333333333334	5.551e-17	nan	nan



kachow

3

a

b

c

d

e

f